Semantic Book Recommender Project Guide

Project explanation and interview preparation

This handbook explains the local semantic book recommender from data preparation to the Gradio interface. It is intended for a college project demonstration, code walkthrough, and interview preparation. The system retrieves existing books using pretrained transformer models; it does not generate books, train a new large language model, or require a paid API.

Read sections 1 through 7 while following the code. Use sections 8 through 12 to prepare your demonstration and interview answers. Run results belong in `reports/validation.md` and `reports/evaluation.json`, where they can be updated without changing the conceptual explanation.

Contents

1 Problem and project scope

2 Architecture and file walkthrough

3 Dataset and cleaning decisions

4 Models and mathematical intuition

5 Retrieval and mood ranking

6 Reliability and reproducibility

7 Setup and troubleshooting

8 Evaluation and evidence

9 Demonstration and resume discussion

10 Interview questions on fundamentals

11 Interview questions on implementation

12 Interview questions on evaluation and tradeoffs

References

1 Problem and project scope

A reader often knows the kind of story they want but cannot name a book. A query such as “an unlikely friendship that helps someone rebuild their life” may not share exact words with a useful book description. The project turns that query into an embedding and compares it with stored embeddings of book blurbs.

The user can constrain results to fiction or non-fiction and prefer a mood. The fiction category is predicted with a zero-shot classifier. Emotional tone is estimated from description text and used only to reorder a semantically relevant candidate set. Gradio presents the shortlist, authors, descriptions, and numerical scores.

The original learning reference is Jodie Burchell's freeCodeCamp semantic book recommender tutorial. The implementation retains the data exploration, vector search, text classification, emotion analysis, and dashboard stages. Local embeddings replace a paid embedding service. Additional work includes category filtering inside retrieval, stable identifier mapping, resumable predictions, a persistent index, automated tests, and an evaluation workflow.

The correct technical description is a content-based semantic recommendation system using pretrained transformer encoders. It is not collaborative filtering because it has no user-item interaction matrix. It is not a generative RAG application because there is no language-model response-generation step. Calling it “LLM semantic book recommender” matches the tutorial title, but explain these distinctions when asked.

2 Architecture and file walkthrough

The offline path is raw CSV → cleaned books → category predictions → emotion probabilities → embeddings → Chroma index. The online path is query → query embedding → category-filtered vector retrieval → optional mood reranking → dashboard results. Expensive classification and emotion inference happen offline, not on every search.

`book_recommender/config.py` centralizes paths and model IDs. Paths are based on the repository directory, so imports do not depend on a particular notebook working directory. Hugging Face weights are cached inside `.cache/huggingface`. No key or hosted inference endpoint is configured.

`data_exploration.py` validates and cleans the catalog, writes the cleaned CSV, and produces an audit report and plots. `text_classification.py` converts descriptions into broad category predictions. `sentiment_analysis.py` creates seven emotion columns. `checkpoints.py` stores content-addressed predictions to avoid recomputing completed work after interruption.

`vector_search.py` contains a small LangChain embedding adapter, index construction, manifest checks, retrieval, and ranking. `scripts/build_pipeline.py` runs these stages in the right order. `scripts/download_data.py` retrieves the public dataset, while `scripts/evaluate.py` runs sanity checks and optionally compares semantic search with TF-IDF.

`app.py` defines the Gradio interface and formats recommendations as HTML cards. Catalog text is escaped before rendering so markup in a book title or description cannot become executable HTML. The five numbered notebooks explain the same operations using the shared modules rather than duplicating an entire pipeline in notebook cells.

3 Dataset and cleaning decisions

The source is Dylan Castillo's 7k Books with Metadata dataset from Kaggle. Its fields include ISBN, title, author, description, source categories, publication year, ratings, and thumbnail URLs. The project uses descriptions for semantic features. Ratings are not part of ranking: popularity is a different recommendation signal and could bias results toward already well-known books.

ISBNs are read and written as strings. A numeric identifier can lose leading zeros or acquire scientific notation. Each accepted ISBN must contain 13 digits, each title must be nonempty, and each normalized description must contain at least 20 words. This checks the identifier's shape, not its mathematical ISBN check digit. Whitespace is collapsed and duplicate ISBNs are removed while retaining the first eligible row.

Missing authors become “Unknown author” and missing source categories become “Uncategorized.” A missing cover does not make a description unsuitable for search. The dashboard uses text cards, which also avoids depending on external cover-image hosts during a demonstration.

The cleaning report records input size, missing values, invalid identifiers or titles, short descriptions, duplicate ISBNs, and output size. Those counts are computed, not typed into the README as assumptions. Different editions can have different ISBNs but almost identical descriptions; identifier deduplication does not solve semantic duplication.

Twenty words is an engineering heuristic. Increasing the threshold may remove noisy entries but also discard useful concise blurbs. A good extension is to compare thresholds against a held-out query set. Another limitation is that English-focused models are not guaranteed to handle every language represented in the source data.

4 Models and mathematical intuition

The embedding model is `sentence-transformers/all-MiniLM-L6-v2`. It converts text into 384 floating-point features. Individual dimensions are not named concepts such as “romance” or “science”; meaning is distributed across the vector. The model was pretrained for sentence representations and is used here for inference without fine-tuning.

The adapter normalizes each embedding to unit length. Cosine similarity is the dot product divided by the product of vector norms. For normalized vectors, it becomes the dot product. Chroma returns cosine distance in this project, so displayed similarity is one minus distance. Similarity is not a likelihood of reader satisfaction.

MiniLM has a finite input window, normally 256 wordpieces in this configuration. A long description may be truncated during embedding. The simple baseline stores one vector per book. A more complete design could embed overlapping chunks and aggregate multiple hits back to one ISBN, but that adds indexing cost and duplicate-hit handling. The current limitation is documented rather than hidden.

The fiction classifier is `MoritzLaurer/deberta-v3-base-zeroshot-v2.0`, a natural language inference model specialized for zero-shot classification. A title plus description is the premise. Candidate labels are inserted into “This book is {}.” The pipeline compares entailment evidence for Fiction and Non-fiction, selecting one label. The source genre is not supplied to the model, so the classification is based on text rather than a copied category rule.

Zero-shot does not mean the model never saw labelled data. It means this project does not train it on labelled examples for this book categorization task. Its winning label score is relative to the candidates and can be overconfident. Values below 0.65 are marked for review as a heuristic, not as a proven error boundary.

The emotion model is `j-hartmann/emotion-english-distilroberta-base`. Its labels are anger, disgust, fear, joy, neutral, sadness, and surprise. Description sentences are split into chunks capped at 100 words, each chunk is scored, and probabilities are averaged by emotion. Mean aggregation estimates an overall blurb tone; maximum aggregation would instead emphasize any intense passage. Probabilities approximately sum to one after averaging.

The model judges the supplied description, which may be promotional or incomplete. It cannot certify how a reader will feel. Suspenseful uses fear and calm uses neutral as approximate mappings. These models run on the local CPU, so there are no per-request API fees. Internet and local disk/compute are still needed for the initial setup.

5 Retrieval and mood ranking

Each indexed LangChain Document contains one description. Its metadata stores the book identifier and predicted fiction category. Explicit metadata is safer than inserting the ISBN into the description and later extracting it by splitting text. The query is embedded by the same model and normalization used during indexing.

When the user selects a category, that constraint is sent as a Chroma metadata filter. This avoids retrieving a small mixed-category set first and then losing most results during a later filter. The system retrieves at most the larger of 50 and four times the requested count, bounded by the eligible catalog size.

Each hit is resolved to its catalog row using its metadata ID. This preserves retrieval order. Simply selecting all matching rows from the original DataFrame would preserve the catalog's row order instead, accidentally discarding the semantic ranking.

For Any tone, ranking follows cosine similarity. For a selected emotion, the system first rescales cosine similarity from minus one through one into zero through one. It then computes ranking_score = (1 - weight) × normalized_similarity + weight × emotion_probability. A default weight of 0.25 is an initial design choice, not a learned optimum.

For example, book A has similarity 0.80 and joy 0.10; book B has similarity 0.70 and joy 0.80. Their normalized similarities are 0.90 and 0.85. At weight zero A ranks first. At weight 0.25 their combined scores are 0.7000 and 0.8375, so B moves ahead. This makes the relevance-versus-mood tradeoff explicit.

Reranking is limited to the semantic candidate pool. A very joyful but irrelevant book elsewhere in the catalog cannot jump into the shortlist solely due to its emotion score. Still, blending uncalibrated scores can harm relevance, especially at high mood weights. Human evaluation should guide later changes. Exact ties are broken using similarity and then ISBN for deterministic results.

6 Reliability and reproducibility

Inference checkpoints are newline-delimited JSON records keyed by the text, model identifier, and stage settings. Cached predictions are reused. New or changed descriptions get new keys. Each batch is flushed to disk; a truncated final line from an interrupted process is ignored on the next run. Changing a model ID or inference settings invalidates the corresponding cached entries.

The index collection name includes a fingerprint of the enriched CSV and a hash of the embedding model ID. A manifest records the collection, model, catalog size, data checksum, and cosine metric. It is published only after all records have been inserted. Repeated builds use stable book IDs and upserts rather than duplicating records.

On startup, the recommender checks the catalog fingerprint, model identity, and stored record count. A changed catalog or incomplete index produces a rebuild instruction. Older collections are retained to avoid destructive replacement during a failed build. They consume additional disk space; rebuilds are not an automatic garbage-collection system.

Model IDs can refer to upstream repositories whose weights change. For strict reproducibility, pin the Hugging Face revision hash as well as Python package versions and dataset checksum. The current cache keys include model names but not an immutable revision. Keep the generated dependency lock snapshot and validation report with the project when sharing measured results.

The Gradio process loads only the embedding model and persisted index. The first query may be slower because initialization is lazy. Subsequent requests reuse the same engine. The server binds to localhost with public sharing disabled. There is no authentication or production deployment design; running it publicly would require a separate security and capacity review.

7 Setup and troubleshooting

Use 64-bit Python 3.11 or 3.12 and create a virtual environment in the project. Install the CPU build of PyTorch first, then requirements.txt. Download the dataset and build a small 200-book catalog to verify the pipeline before processing every eligible book. The README contains exact PowerShell commands that do not require environment activation.

The normal sequence is: `python -m scripts.download_data`, `python -m scripts.build_pipeline --limit 200`, then `python app.py`. These commands assume the virtual environment's Python is active; on Windows the explicit alternative is `.venv/Scripts/python.exe`. Build the full catalog by omitting `--limit`. Classification checkpoints allow completed work to be reused.

Run stages individually with `--stage clean`, `--stage classify`, `--stage emotions`, or `--stage index`. A stage consumes the previous stage's saved file. The limit applies to classification; running emotions or index later uses whatever rows were saved by classification. Close and restart the dashboard around a rebuild so it reloads the new catalog.

If Kaggle's public download fails, save the dataset's books.csv manually under data/raw. If model downloads fail, check internet access and rerun; cached files can be reused. Do not enable HF_HUB_OFFLINE until weights are present. After downloading, setting HF_HUB_OFFLINE=1 forces model loading from local cache.

If RAM is limited, reduce batch size to four and start with a subset. If search reports a stale catalog, rerun the index stage. If the port is occupied, stop the previous server or choose another port. If suggestions are poor, inspect actual descriptions and test clearer English queries; an operational pipeline does not guarantee every recommendation is useful.

8 Evaluation and evidence

Software tests check missing descriptions, identifier preservation, duplicate removal, chunk boundaries, mood reranking, cache invalidation, and HTML escaping. A Chroma integration test uses deterministic tiny embeddings to exercise real persistence, filtering, repeat builds, identifier lookup, and stale-data rejection. These tests prove specific behaviors, not pretrained model quality.

The evaluation script samples up to 20 descriptions with a fixed seed, queries using up to the first 1,000 characters, and checks whether the original book ranks first. This description self-retrieval metric is a sanity check. It is easier than genuine recommendation and may be affected by duplicate blurbs. The script also records median and 95th-percentile query times, excluding initial model loading.

For recommendation quality, fill the relevant ISBN lists in data/evaluation_queries.json using independent human judgments. Pool candidate results from semantic search and a TF-IDF baseline, hide system identities, and apply a consistent relevance rubric. Keep a separate test query set if choosing models, thresholds, or mood weights from development results.

Precision@5 is the count of relevant returned books divided by five. Recall@5 divides that count by the relevant set size. Binary nDCG@5 discounts each relevant result by log2(rank + 1) and normalizes against the best possible ordering. Incomplete relevance sets can make recall misleading. Ten template queries are a learning starting point, not a comprehensive benchmark.

The evaluator rejects empty relevance labels rather than reporting a fabricated score. Classification needs its own human-labelled sample, per-class precision and recall, and a confusion matrix. Emotion analysis can be checked with a human-rated subset but remains subjective. The model card's original benchmark is not this project's measured accuracy.

9 Demonstration and resume discussion

For a short demonstration, open the dashboard with the prebuilt catalog. Search for an idea such as a mysterious disappearance. Show the title, author, and description of a result. Select Fiction, then change the emotional tone and explain that mood only reranks the semantic shortlist. Expand the score panel and state that similarity is not a percentage accuracy.

Next, show the folder structure and one notebook. Point to the offline preprocessing stages, the LocalEmbeddings adapter, metadata filtering, and stable-ID lookup. Finish by showing the test and evaluation reports. If asked about scale or accuracy, use the actual catalog size and measured values, and identify what still needs human evaluation.

A suitable 60-second explanation is: “I built a content-based book recommender that accepts natural-language descriptions. I used local MiniLM embeddings through LangChain and stored them in Chroma. I predicted fiction categories with zero-shot NLI and extracted seven emotions from description chunks. The Gradio interface applies category constraints during retrieval and reranks the closest books by mood. I separated preprocessing from serving, added persistence and tests, and included an evaluation comparison with TF-IDF. I used pretrained models rather than training an LLM.”

Resume wording should reflect work you have run and can explain. Suitable bullets are: “Built a semantic book search pipeline using LangChain, local transformer embeddings, and persistent Chroma retrieval”; “Added zero-shot fiction categorization and seven-emotion description scoring”; and “Developed a Gradio dashboard with category filtering, adjustable mood reranking, and interpretable result scores.” Add catalog size or latency only from an actual verified run.

Credit the tutorial as the foundation. Explain which improvements you made and why. Do not claim proprietary model training, perfect mood detection, personalized recommendations, a deployed production service, or an unmeasured accuracy improvement. Dates and deployment claims in a resume should reflect your actual work history.

10 Interview questions on fundamentals

1 What problem does the project solve

It lets a reader search for books using an idea or theme without knowing titles or exact keywords. It uses description similarity, an optional broad category constraint, and an emotion preference. It does not use past ratings or user behavior.

2 Why use semantic search instead of keyword matching

Embeddings can place related expressions near each other even when they use different words. Keyword matching is valuable for exact names, rare terms, and interpretability, so I included a TF-IDF baseline rather than assuming semantics always wins.

3 What is an embedding

It is a learned numerical representation of text. Here it is a normalized vector with 384 dimensions. Similar meanings should have nearby representations, but this is a learned approximation and can fail for negation, unusual domains, or very specific constraints.

4 Why must queries and documents use the same model

Their coordinates must describe the same learned representation space. Different model outputs are generally not comparable even if their vector dimensions happen to match. Changing the embedding model requires rebuilding stored vectors.

5 How does cosine similarity work

It measures the angle between two vectors using their normalized dot product. In this project vectors are unit-normalized, so the dot product is enough. Chroma returns cosine distance and the code converts it to similarity with one minus distance.

6 What does LangChain contribute

It provides the Document, Embeddings, and Chroma integration interfaces used to connect the components. It is not the embedding model or the vector database. The adapter makes local Sentence Transformer inference fit LangChain's embedding interface.

7 Why choose Chroma

It offers a convenient local persistent vector store with metadata filtering and a LangChain integration. For a small corpus brute-force NumPy similarity could also work. Chroma makes persistence and later growth easier, but adds dependencies and index management.

8 What is zero shot classification

It applies a pretrained model to labels without training on labelled examples from this task. The NLI pipeline treats each candidate label as a hypothesis and scores its compatibility with the book text. The underlying model itself was trained on other labelled data.

9 How is emotion analysis different from sentiment analysis

Sentiment often means positive, negative, or neutral polarity. This project distinguishes seven emotion classes. Fear and sadness may both be negative but provide different signals for mood-based discovery.

10 Is this RAG or collaborative filtering

It performs the retrieval component often used in RAG, but no generator answers from retrieved context. It is content-based recommendation. Collaborative filtering would require user-item interactions and a different modeling approach.

11 Interview questions on implementation

11 Why keep ISBNs as strings

They are identifiers, not quantities. Numeric conversion can lose leading zeros or change formatting. A stable string ID links each vector result to exactly one catalog record without parsing the description text.

12 Why filter the category inside Chroma

Filtering after a small top-k search can leave too few eligible books and miss good category-specific matches outside that initial set. A metadata filter constrains the search before the candidate shortlist is formed. Its quality still depends on predicted category correctness.

13 How do you preserve recommendation order

I iterate through Chroma's returned hits and resolve each metadata ID to its DataFrame row. Selecting all IDs with an isin mask would retain the DataFrame's original order, potentially losing the retrieval ranking.

14 Why average emotion scores over chunks

It represents the overall description rather than its single strongest emotional sentence. It also lets later sentences contribute despite token limits. Maximum pooling would be a different choice favoring intense moments, which should be compared using human judgments.

15 What happens to long descriptions

Emotion analysis splits them into bounded sentence chunks, with a final token truncation safeguard. Embedding currently uses one vector and may truncate at MiniLM's limit. Chunked embedding with book-level aggregation is a known improvement to evaluate.

16 How does mood influence ranking

Within the semantic shortlist, I blend rescaled cosine similarity and the selected emotion probability. Weight zero preserves semantic order; larger values favor mood. These scales are not calibrated, so the default is a heuristic and needs evaluation.

17 How do you avoid repeating expensive computation

I cache classification and emotion predictions using hashes of model ID, text, and settings. Completed batches are reused after interruption. A different text or setting produces a new cache key. Immutable model revisions would improve strict reproducibility further.

18 How do you detect a stale index

The index manifest stores the enriched CSV's checksum and embedding model identity. Loading compares them with current inputs and checks the stored record count. A mismatch stops search with a rebuild instruction rather than silently serving inconsistent results.

19 What happens when a user submits an empty query

The search function validates it and returns a clear Gradio error. It also limits query length and checks category and result-count values. Catalog text is HTML-escaped when cards are rendered.

20 Why separate preprocessing from the app

Classification, emotion scoring, and document embedding are expensive catalog-level tasks. Search only needs query embedding and retrieval, so the dashboard loads the embedding model and stored index. This reduces per-query work and makes failures easier to debug.

12 Interview questions on evaluation and tradeoffs

21 How would you measure recommendation quality

I would collect human relevance labels for representative queries and compare semantic retrieval with TF-IDF using precision@5, recall@5, and nDCG@5. I would keep held-out queries for final reporting and make clear that relevance judgments may be incomplete or subjective.

22 Why is self retrieval not accuracy

Searching with a book's own description is an easy consistency check, not a realistic reader query. It verifies the embedding/index/ID path and can catch corruption. It does not prove that results satisfy varied human preferences.

23 How would you evaluate the fiction classifier

Manually label a representative sample, include ambiguous cases, and calculate a confusion matrix and precision/recall by class. Source genres can help sample the data but should not automatically become unquestioned ground truth. Inspect mistakes before changing models or thresholds.

24 What are the most important limitations

Short or promotional blurbs may misrepresent books; long embedding inputs are truncated; category predictions can exclude relevant books; emotion proxies are imperfect; and the system lacks user personalization. English model bias and duplicate editions can also affect results.

25 How would you improve relevance

Start with error analysis and human labels. Try hybrid lexical-plus-semantic retrieval, chunked embeddings, and a cross-encoder reranker for a small shortlist. Compare against the current baseline while measuring added latency and memory costs.

26 How would you scale to a million books

Use incremental ingestion, immutable model versions, batched embedding, an appropriately configured approximate nearest-neighbor service, and monitoring. Separate offline workers from online serving, measure memory and recall/latency tradeoffs, and add caching and concurrency controls. This local prototype has not demonstrated that scale.

27 What does free mean here

The models and inference run locally without a billed API or subscription. Downloads, electricity, storage, and hardware still have costs. Model and dataset licensing conditions remain relevant, especially if redistributing artifacts or deploying commercially.

28 Did you train a large language model

No. I integrated pretrained transformer models and built the data, indexing, ranking, interface, and testing pipeline. Fine-tuning would require suitable labelled data, a training setup, and a separate held-out evaluation.

29 What would you change if you had more time

First gather reliable human judgments, then test hybrid retrieval and chunked embeddings. Add explicit model revision pins and stronger data versioning, measure category uncertainty, and consider an Unknown category. Personalization would be a separate feature requiring user feedback and consent.

30 What was your main engineering contribution

I turned a learning workflow into an organized local project: reusable stages and notebooks, free inference, metadata-based filtering, rank-preserving ID mapping, resumable preprocessing, persistent indexing, tests, and an explainable Gradio interface. I can show the code and measured reports for each implemented behavior.

References

Tutorial: https://www.youtube.com/watch?v=Q7mS1VHm3Yw

Companion code: https://github.com/t-redactyl/llm-semantic-book-recommender

Dataset: https://www.kaggle.com/datasets/dylanjcastillo/7k-books-with-metadata

Embedding model: https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

Zero shot model: https://huggingface.co/MoritzLaurer/deberta-v3-base-zeroshot-v2.0

Emotion model: Jochen Hartmann, Emotion English DistilRoBERTa base, 2022. https://huggingface.co/j-hartmann/emotion-english-distilroberta-base

LangChain integration: https://docs.langchain.com/oss/python/integrations/vectorstores/chroma

Gradio: https://www.gradio.app/docs